

WikiSkill 小白指南

小模型打败大模型？不微调、不重训，让技能和一本“活百科”共同进化

arXiv:2608.27454 • Google Research • 2026-08-27

论文: WikiSkill: Compiling Agent Experience into Persistent Knowledge for Skill Evolution

作者: Liyan Tang, Cyrus Rashtchian, Chun-Sung Ferng, Andrew Tomkins, Da-Cheng Juan, Tu Vu

本指南: 疑问式导读 + 零 API 方法层 toy 复现 (F:\reminder\wikiskill\exercise, 4 实验全跑通)

0

一句话说清

给 AI 智能体配一个 **只会越来越厚的“维基百科” (wiki 层)** 和一个 **可以整体换血重来的“技能手册” (skills 层)**：智能体干活留下原始轨迹，维护员把经验沉淀进 wiki，提案员照着 wiki 写新技能，验收员试跑后 **只保留让成绩变好的技能，但 wiki 永远不回滚**。转几圈之后，**9B 小模型 + 技能 = 47.4% > 27B 大模型裸考 39.4%**——权重一个参数没动。

查证结论 (诚实边界)：论文真实 (arXiv HTML 全文已读、多源交叉确认)。官方没有放代码，网上只有第三方复现；本指南实验是**方法层 toy 复现**：用确定性脚本模拟“模型”，验证机制本身，不复现论文分数。

1

WikiSkill 到底是什么？和推文说的"颠覆性"是一回事吗？

WikiSkill 是一个**技能演化 (skill evolution) 框架**：让智能体在反复做题中，把经验"编译"成两种持久资产——知识 (wiki) 和技能 (skills)，从而**不改模型权重就提升能力**。它不训练模型、不微调，全靠外挂的持久知识层。

"颠覆性"要打折听：它不是让小模型凭空变强，而是**用"外挂经验"换"参数规模"**——本质上是一种极省钱的能力搬运术。方向确实诱人，但论文自己列了 4 条局限（见 Q12），离"颠覆"还有距离。



2

"小模型打败大模型"的数字是怎么来的？

论文用 5 个基准（LiveMath 数学 / SealQA 搜索 / SpreadSheetBench 表格 / OfficeQA 长文档 / ALFWorld 具身交互）× 5 个模型。最抓眼球的对照：

Qwen-3.5-9B + WikiSkill 47.4% vs Qwen-3.6-27B 裸考 39.4%

参数量差 3 倍，分数反超 8 个点——权重零改动

而且**模型越大，WikiSkill 增益越大**：4B / 9B / 27B 平均提升分别为 **+12.3 / +17.5 / +23.9** 分（SpreadSheet 上 27B 更是 +40.9）。技能和规模是**互补**关系，不是穷人的 substitutes。

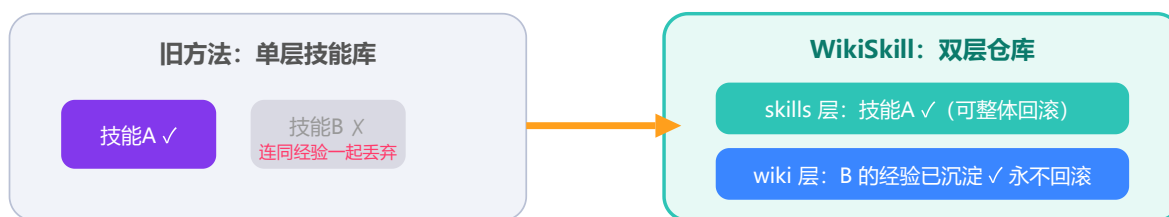
模型	裸考	Trace2Skill	EvoSkill	SkillOpt	WikiSkill
Qwen-3.5-4B	26.2	32.1	33.7	35.2	38.5
Qwen-3.5-9B	29.9	36.7	42.3	40.2	47.4
Qwen-3.6-27B	39.4	47.3	53.3	50.7	63.3
Gemma-4-31B	41.3	45.8	43.4	49.1	54.9
Gemini-3.5-Flash	49.5	55.6	56.1	55.9	68.1

五个模型 × 四个对手，WikiSkill 平均分全线第一（ALFWorld 上 Gemini 因验证满分提前终止各法并列）。

3 之前的技能演化方法差在哪？

Trace2Skill / EvoSkill / SkillOpt 这批前作都在做同一件事：**从执行轨迹里提炼"技能"**。它们的共同盲区是——**技能被门控拒绝时，背后提炼出的"理解"也跟着丢了**。一条经验要么变成技能存活，要么整个蒸发，学习没有"复利"。

WikiSkill 的关键一步是把仓库拆成两层：**wiki (理解，永不回滚) + skills (可执行步骤，严格验收)**。技能被拒了，但这次踩坑学到的知识已经写进 wiki，下轮提案员还能从 wiki 里重新长出更好的技能。**经验从"一次性消耗品"变成"复利资产"**。



4 三层工作区各存什么？

workspace/	
├─ raw/	① 原始执行轨迹：每道题怎么做的、对没对。只追加、不可变（案卷底档）
├─ wiki/	② 持久知识层（只能由 Wiki Maintainer 写）
│ ├─ patterns/*.md	模式页：一类坑的根因分析与正确做法（如 adddays.md"日期要用真日历"）
│ ├─ logs.md	演化日志：每轮发现了什么（流水账）
│ ├─ skill-impact.md	技能提案审计：谁提的、改了什么 diff、验收分数、收还是拒
│ └─ index.md	索引：所有模式页的一句话目录（改了页就同步改索引）
└─ skills/	③ 可执行技能层：SKILL.md（怎么做）+ PURPOSE.md（为什么）
	可以整批换掉、回滚——像一叠可以随时重印的说明书

类比本仓库 vault：[[第二大脑LLMWiki]] 说"raw 人写只读、wiki 归 LLM 维护"——WikiSkill 把这套搬进智能体：raw 是**自动留下的案卷**，wiki 是**消化后的理解**，skills 是**照着能直接上手的操作卡**。三层权限不同、寿命不同：raw 和 wiki 只增不删，skills 随时可以回炉。

5

四个角色怎么组成一台"进化机器"? (Algorithm 1)



每轮五步: 训练 rollout → Wiki 维护 (吃采样轨迹子集, 防上下文爆仓) → 技能提案 → 候选集上验证 → 门控。验证分数超过历史最优才接受整批候选技能, 否则技能回滚到上一代; **无论收拒, wiki 都保留**。每轮验证后, 外层程序自动往 `skill-impact.md` 追加一条审计 (提案元数据 + unified diff + 分数 + 收/拒)。

6

为什么推理 Agent 训练时偏偏不许读 wiki?

反直觉但极重要: 训练 rollout 时推理 Agent 只拿到技能全文注入系统提示, wiki 对它封锁。消融实验证明让它读 wiki 反而更差:

Proposer 读 wiki (默认) 63.7% vs 推理时也读 wiki: 60.9%

LiveMath 上掉得更狠: 72.6 → 64.8

原因: **答案能从 wiki 直接抄到, 轨迹就没有信息量了**——模型"开卷抄作业", 维护员反而提炼不出真经验。类比: 教室里先闭卷考 (暴露真实弱点), 再由老师把错题整理进错题本 (wiki), 下次带的是"解法卡" (skills) 而不是整本错题本。

关联: [[三层记忆]] / [[记忆四层模型]] 讲"经验怎么分层压缩"; WikiSkill 补了一刀——**压缩的原料必须来自"真实暴露的失败"**, 喂料端被污染, 整条流水线退化。

7

Wiki Maintainer: 图书管理员怎么"补丁式"维护 wiki?

每轮拿到两样东西：当前 wiki 全状态 $W(k-1)$ + 采样的轨迹子集（采样是为了 **不撑爆上下文窗口**）。它做三件事：

1

根因分析

对失败任务问"为什么错"，对通过任务提炼"为什么对"

2

补丁式编辑

新建模式页，或对已有页做 **追加 / 替换 / 插入文本片段**——注意：没有自动 merge，也没有 prune（剪枝），页数只增不减（局限 3：长期跑可能臃肿）

3

同步索引和日志

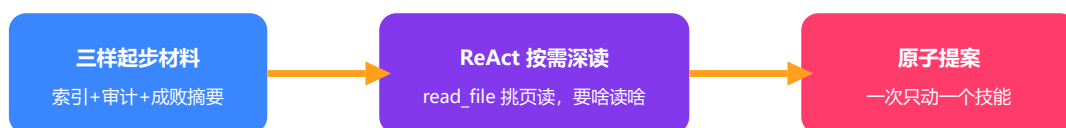
模式页变了就更新 index.md；每轮往 logs.md 追加发现摘要

论文里真实长出的模式页如 `take-examine-move-loop.md`（ALFWorld 的"取-查-动"循环）、`multi-operation-loop.md`（表格多步操作）。

8

Skill Proposer: 侦探式按需检索，不是"一次性塞全库"

提案员起步只拿三样轻量材料：**wiki 索引、skill-impact 审计、全部训练任务的成败摘要**。然后它像侦探一样 **用 read_file 工具自己决定深挖哪几页模式页、翻哪几条原始轨迹**，最后产出**原子提案**——一次只改一个技能。



这和 [[RAG检索增强生成]] 的"先向量检索 Top-k 再喂给模型"不同——**检索权在 agent 手里，是主动查阅不是被动投喂**；和 [[分层按需加载]] 的精神一致：索引常驻，正文按需。

9

Gating: 为什么技能可以整批回滚, wiki 却一刀不动?

门控规则简单粗暴: 候选技能集上验证集, 分数必须严格超过历史最优, 否则整批回滚。这保证了技能集永远单调不退步。但 wiki 被刻意豁免——因为 wiki 存的是理解, 不是行为: 这次提案失败了, 但“我们搞清楚了这类题为什么难”这件事永远为真。

我们的 toy 实验 ex2 实证了这一点:

坏技能候选 `val=62.5%` < 最优 `87.5%` → 门控拒绝, `skills` 回滚成功
`wiki` 页数 4→4、`logs` 只增不删: `wiki` 层永不回滚
 删掉 `adddays` 技能后重演化: 提案=空 (`skill-impact` 审计拦下重复提案)
 但 `wiki/patterns/adddays.md` 仍在: 知识活着

代价也要知道: 严格门控会错杀“中性但长期有价值”的提案 (分数没涨就被拒) ——论文局限 2 原文承认这一点。

关联: [[发布交换与健康门回滚]] (发布是切换不是修改)、[[Checkpoint存档与持久执行]] (回滚到快照) ——WikiSkill 把这套工程纪律第一次用在“技能集”上: `skills` 像蓝绿部署的实例, `wiki` 像只增不删的事务日志 ([[JSONL事件日志与折叠模型]])。

10

skill-impact.md: 一份让提案员“不二次踩坑”的审计账本

每次验证结束, 外层 harness (不是 LLM) 程序化追加一条记录: 提案元数据、目标技能名、修改的 unified diff、验证分数、Accept/Reject。它同时服务两端:

读者	用途
Skill Proposer	看到“这条 diff 上轮被拒+为什么”, 不再重复提交同款失败修改 (我们 ex2 的审计拦截就是它的玩具版)
人 / 外层系统	完整审计轨迹: 技能是怎么一步步进化成现在这样的, 每次转弯有据可查

本质是 [[事实与判断分离]] 的又一实例: 收/拒的判断由门控做出, 但记录由确定性代码写入——LLM 可以读账本, 不许改账本。

演化出的技能可以直接搬给别的模型（Table 2 精选）：

场景	裸考	自演化	用别家技能
27B 的技能 → 9B @SpreadSheet	24.3	33.6	50.5
27B 的技能 → Gemma-31B @LiveMath	33.9	56.7	73.7
9B 的技能 → 27B @ALFWorld	—	63.4	70.2
负迁移：4B 的技能 → Gemini-Flash @SpreadSheet	—	50.5	18.1（暴跌！）

负迁移的机制值得记住：4B 弱模型总结出的"低层 workaround"（为绕开自身局限设计的笨办法）注入强模型后，反而把强模型本来就优雅的解法捆住了，碎片化诊断还耗光了交互预算。同款技能给 27B 则升到 63.4——**技能不是越通用越好，要匹配受体的能力档位。**

我们的 ex3 也复刻了这个现象：B 模型全量迁移 A 的技能 50%→87.5%（正迁移），但单迁 A 的闰年"简化规则"技能，把本来全对的 B 带错世纪年（leap 2/2 → 1/2，负迁移实证）。

12

我能拿它干什么？论文自己承认哪些坑？

论文四条局限（原文承认）：① 技能是全量注入 prompt 的，没评测技能检索/触发；② 严格门控会错杀中性但长期有价值的提案；③ wiki 没有自动剪枝，长期跑会臃肿；④ 没覆盖数百步/数小时的超长程任务，在线适应留作未来工作。

给你的 6 个可动手用途：

1

给个人 Agent 装外挂成长环

不需要 API 微调权限：把 Claude Code 的技能包迭代按"wiki 沉淀 + skills 验收"双轨管理（对照 [[AgentSkills技能包]]、[[团队Harness的Git原生分发]]）

2

评审"自我改进 Agent"产品的问题清单

它有永不回滚的知识层吗？技能怎么验收？有没有提案审计？——三问筛掉大半PPT项目

3

技能迁移选型

给弱模型配技能 vs 付大模型账单：本指南 Q2/Q11 的数字就是谈判筹码

4

警惕负迁移

别把弱模型总结的低层 workaround 塞给强模型（Q11 的 50.5→18.1 惨案）

5

审计账本模式复用

skill-impact.md 的"程序化追加、LLM 只读"纪律可搬到任何 AI 决策留痕场景

6

自己跑一遍 toy 复现

本仓库 exercise\wikiskill_toy.py，零 API 零依赖，4 个实验 10 秒跑完

E

动手实验：零 API 方法层 toy 复现（已全部真跑通）

没有 GPU 也没有 API key 怎么验证一篇论文？——把“模型”换成确定性 stub，只验证机制。两个 stub 模型各带系统性错误（权重全程冻结），Wiki Maintainer / Skill Proposer / Gating 全部按 Algorithm 1 用脚本实现。

1 运行

```
cd F:\reminder\wikiskill\exercise && python wikiskill_toy.py
```

（纯标准库，10 秒）

2 ex1 演化循环

ModelA 基线 val=12.5% → 3 轮演化后 87.5%。模型函数一行未改，提升全部来自技能层；leap 题仍错 1 题——技能自带的简化闰年规则漏了世纪年，正是论文“低层 workaround”的活标本。

✓ 预期结果：12.5% → 87.5%，技能数 4

3 ex2 污染测试

故意注入坏技能（系数×2）：门控验出 62.5% < 87.5% 拒绝并回滚 skills；wiki 页数只增不删；审计账本拦下重复提案。

✓ 预期结果：skills 回滚、wiki 保留、审计生效

4 ex3 跨模型迁移

ModelB 裸考 50% → 全量迁移 A 的技能 87.5%（正迁移 miles/adddays）；但 A 的闰年简化规则把本来全对的 B 带错世纪年（leap 2/2 → 1/2，负迁移实证）。

✓ 预期结果：正负迁移并存，复刻论文 Table 2 形态

5 ex4 复杂度记账

训练集 40 → 80 题翻倍：Wiki Maintainer 调用 3 → 3、Proposer 调用 3 → 3（优化器侧不随数据量涨），只有推理 rollout 120 → 240。

✓ 预期结果：优化器组件调用数与 N 无关（全批模式主张；注：论文附录 D.2 的精确公式在 HTML 版中被截断，此处验证的是正文可推断的每轮开销结构）

诚实边界：stub 替代了真实 LLM，验证的是**方法机制**（三层工作区 / 门控只回滚技能 / wiki 永续 / 审计 / 调用结构），不是论文分数；论文的“LLM 写 wiki、LLM 提技能”在这些位置被确定性规则替代。真实输出全文在 `exercise\output.txt`。



放进你的知识地图



一句话定位：[[第二大脑LLMWiki]] 回答“知识库怎么分工”，WikiSkill 回答“知识库怎么让智能体越用越强”——把“人查维基”变成“维护员写维基 + 提案员读维基 + 门控验技能”的自动进化闭环。